Data & Planning: Model-Based & Offline RL

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

Recap & Today

- Day 7 (*The Reward Problem*) was about getting a *good signal* from the world: exploration (optimism, posterior sampling, curiosity) and designing or learning the reward
- One theme ran through it: what happens **off the data distribution**, where the agent has not been
- Today is about what to *do* with the data you have:
  - **Model-based RL**: learn a model of the world, plan with it, and fix mistakes by collecting more data where the model is wrong
  - **Offline RL**: learn from a *fixed* batch with no new interaction (the same off-distribution problem, with that fix *forbidden*)
- Two halves, one spine: distribution shift, and what you are allowed to do about it

Model-Based and Model-Free RL

- Model-Free RL
  - No model
  - Learn value function (and/or policy) from experience

Model-Based and Model-Free RL

- Model-Free RL
  - No model
  - Learn value function (and/or policy) from experience
- Model-Based RL
  - Learn a model from experience
  - Plan value function (and/or policy) from model

Model-Based and Model-Free RL

- Model-Free RL
  - No model
  - Learn value function (and/or policy) from experience
- Model-Based RL
  - Learn a model from experience
  - Plan value function (and/or policy) from model

- So far, we have only looked at model-free reinforcement learning

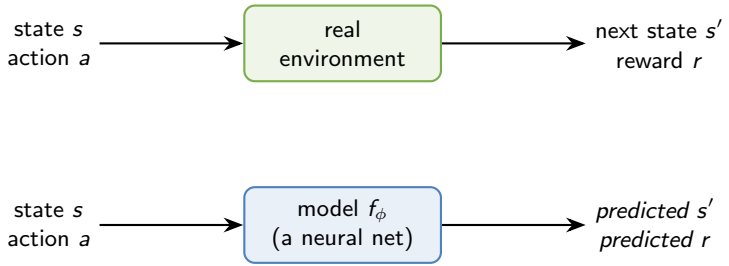
What is a Model?

- A model is a **learned simulator of the environment**
- It is *not* a state, and *not* the real environment itself: it is a *function* (e.g. a neural network) that *imitates* how the environment behaves

What is a Model?

- A model is a **learned simulator of the environment**
- It is *not* a state, and *not* the real environment itself: it is a *function* (e.g. a neural network) that *imitates* how the environment behaves
- You ask it: *"if I take action a in state s, what happens next?"*
- It answers with a predicted **next state** s' and **reward** r, just like the real environment would, but without having to actually be there

What is a Model?



the model *imitates* the environment

- Same inputs, same kind of outputs: the model is a stand-in we can query as often as we like, for free

What is a Model? (Formally)

- The environment is a Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$:
  - $\mathcal{P}$: the true transition dynamics (next-state rule), *unknown*
  - $\mathcal{R}$: the true reward function, *unknown*
- We assume the states $\mathcal{S}$ and actions $\mathcal{A}$ are known
- The model $\mathcal{M}_\phi = \langle \mathcal{P}_\phi, \mathcal{R}_\phi \rangle$ *learns* to approximate the unknown parts: $\mathcal{P}_\phi \approx \mathcal{P}$ and $\mathcal{R}_\phi \approx \mathcal{R}$
- Here ϕ are the model's **parameters**: the weights we train from data
- When the model predicts the next state directly we write $s' = f_\phi(s, a)$, the same f_ϕ from the diagram, used in the algorithm shortly

$$\underbrace{s_{t+1}}_{\text{next state}} \sim \underbrace{\mathcal{P}_\phi(s_{t+1} \mid s_t, a_t)}_{\text{predicted from current state \& action}}, \quad \underbrace{r_{t+1}}_{\text{reward}} \sim \mathcal{R}_\phi(r_{t+1} \mid s_t, a_t)$$

What is a Model? (Formally)

- The environment is a Markov Decision Process $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R} \rangle$:
  - $\mathcal{P}$: the true transition dynamics (next-state rule), *unknown*
  - $\mathcal{R}$: the true reward function, *unknown*
- We assume the states $\mathcal{S}$ and actions $\mathcal{A}$ are known
- The model $\mathcal{M}_\phi = \langle \mathcal{P}_\phi, \mathcal{R}_\phi \rangle$ *learns* to approximate the unknown parts: $\mathcal{P}_\phi \approx \mathcal{P}$ and $\mathcal{R}_\phi \approx \mathcal{R}$
- Here ϕ are the model's **parameters**: the weights we train from data
- When the model predicts the next state directly we write $s' = f_\phi(s, a)$, the same f_ϕ from the diagram, used in the algorithm shortly
- We can also learn just *one* of the two (only dynamics, or only reward); often the dynamics $\mathcal{P}_\phi$ is the hard part

Why Build a Model of the Environment?

- A fair objection: if we can already *act* in the environment, why bother building an approximate *copy* of it?

The Catch with the Real Environment

- The real environment offers only *one* operation: **take an action, see what happens** (once, forwards, at a cost)

The Catch with the Real Environment

- The real environment offers only *one* operation: **take an action, see what happens** (once, forwards, at a cost)
- You *cannot* ask "what would have happened if I had done something else?" (no what-ifs)
- You *cannot* reset it to any state you like
- You *cannot* look ahead without actually committing to the action
- Every single query costs a **real step**: time, money, wear, risk (a real robot can break; a real patient can be harmed)

What a Model Gives You

- A learned model is a **cheap, fast, safe simulator** you can query as often as you like

What a Model Gives You

- A learned model is a **cheap, fast, safe simulator** you can query as often as you like
- **Sample efficiency**: turn a little real data into (almost) unlimited synthetic experience
- **Planning**: imagine the consequences of actions *before* committing to them
- Superpowers the real world denies you: ask counterfactuals, reset to any state, even differentiate through it
- It is trained by cheap **supervised learning**, and is somewhat task-agnostic (reusable across different rewards)

The Trade-off

- No free lunch: the model is only an *approximation* of the real environment
- A *wrong* model can confidently mislead you. We will see exactly this failure (the "cliff") shortly
- Model-based RL trades *real-world cost* for *model error*: worth it when the model is easier to get right than the policy itself

Model Learning

- Goal: estimate the model $\mathcal{M}_\phi$ from logged experience $s_1, a_1, r_1, \cdots, s_T$

Model Learning

- Goal: estimate the model $\mathcal{M}_\phi$ from logged experience $s_1, a_1, r_1, \cdots, s_T$
- This is a **supervised learning** problem: each transition gives us an input (s, a) and the targets to predict (r and s')

Model Learning: Two Things to Predict

- **Predict the reward**: $s, a \rightarrow r$
  - the reward is just a *number*: the model outputs one value and we fit it to the observed reward
  - predicting a number is a **regression** problem

Model Learning: Two Things to Predict

- **Predict the reward**: $s, a \to r$
  - the reward is just a *number*: the model outputs one value and we fit it to the observed reward
  - predicting a number is a **regression** problem
- **Predict the next state**: $s, a \to s'$
  - the *same* action can lead to *different* next states: the environment is often random (stochastic)
  - so a single guess is not enough; we want the whole *distribution* $p(s' \mid s, a)$ over where we might land
  - learning a distribution is **density estimation**

Model Learning: Picking a Loss

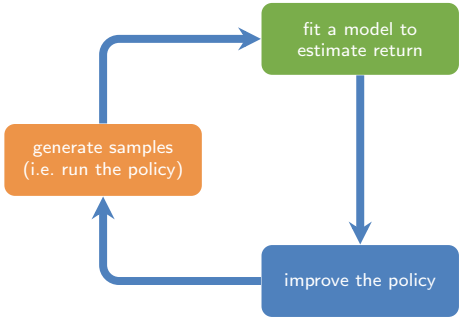
- Each task comes with a matching loss:
  - regression (reward) $\rightarrow$ mean-squared error (MSE)
  - density estimation (next state) $\rightarrow$ maximum likelihood / KL divergence
- Common shortcut: if we *assume* the dynamics are deterministic (one action $\Rightarrow$ one next state), predicting s' collapses to plain regression too
- That is the f_ϕ trained with MSE, $\min_\phi \sum_i \| f_\phi(s_i, a_i) - s_i' \|^2$, that we use in the algorithm next

Model-Based RL: How We Use the Model

- We have the *what* (a learned simulator f_ϕ), the *why* (cheap, safe, plan-able experience), and *how to learn it*. Now: how to **use** it
- Two uses, and they organize the rest of today:
  - **Plan** with the model: pick good actions by looking ahead
  - **Optimize a policy** with the model: train a policy on simulated experience
- One perk we will lean on later: a model lets us *reason about its own uncertainty* (ensembles)

Model-Based RL: The Shared Loop

- Both uses run the same outer loop: collect data, fit the model, act better, repeat:



estimate $p(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})$ (model-based)

supervised learning

$$\min_{\phi} \sum_{i} \| f_{\phi}(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i \|^2$$

optimize $\pi_{\theta}(\mathbf{a} \mid \mathbf{s})$ (model-based)

The bottom step ("improve the policy") is exactly where the two uses *differ*: *plan* good actions directly (next), or *train* a policy π_{θ} on model-generated data (later).

A First Algorithm: Planning

- We take the first option **(planning)**: use the model to choose good actions directly, with no separate policy yet. The simplest version:
  1. Run some policy (e.g. random policy) to collect data $D = \{(s, a, r, s')_i\}$
  2. Learn the model $f_\phi(s, a)$ (weights ϕ) to minimize $\sum_i \| f_\phi(s_i, a_i) - s'_i \|$
  3. Iteratively sample action sequences, run through model $f_\phi(s, a)$ to choose actions

A First Algorithm: Planning

- We take the first option **(planning)**: use the model to choose good actions directly, with no separate policy yet. The simplest version:
  1. Run some policy (e.g. random policy) to collect data $D = \{(s, a, r, s')_i\}$
  2. Learn the model $f_\phi(s, a)$ (weights ϕ) to minimize $\sum_i \| f_\phi(s_i, a_i) - s'_i \|$
  3. Iteratively sample action sequences, run through model $f_\phi(s, a)$ to choose actions
- Step 3 is doing the real work, but *how* do we use the model to "choose actions"? Let us unpack it

How Does the Policy "Plan" with the Model?

- Step 3 said to *"sample action sequences, run through the model to choose actions"*. What does that actually mean?

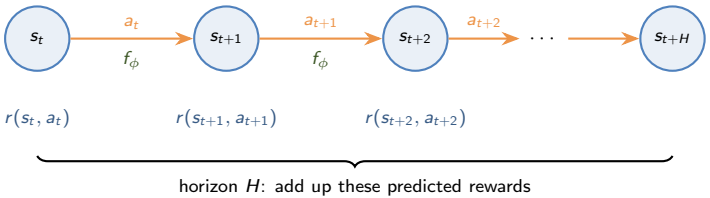
How Does the Policy "Plan" with the Model?

- Step 3 said to *"sample action sequences, run through the model to choose actions"*. What does that actually mean?
- **Planning**: imagine candidate action sequences, use the model to predict what each one leads to, and pick the one with the most reward
- No real environment needed: the model does the imagining. Let us build up the notation, one symbol at a time

Planning: The Symbols

- t: the **current** time step (where the agent is right now)
- H: the **planning horizon** (how many steps ahead we look)
- $a_{t:t+H}$: a whole **sequence** of actions (a_t, a_{t+1}, ..., a_{t+H}); the ":" just means "from t through $t + H$"
- k: a counter for steps into the future, $k = 0, 1, \ldots, H$, so s_{t+k} is "the state k steps from now"

Planning: Rolling the Model Forward



- Start from now (s_t). Feed each state and a chosen action into the model f_ϕ to get the next state. This is "rolling out" the model H steps
- Each step earns a predicted reward $r(s_{t+k}, a_{t+k})$; we add them up

- Try this for many candidate sequences; keep the one whose roll-out scores highest:

$$a^{\star}_{t:t+H} = \underbrace{\arg\max_{a_{t:t+H}}}_{\text{choose the sequence...}} \underbrace{\sum_{k=0}^{H} r(s_{t+k},\, a_{t+k})}_{\text{...with the most predicted reward}}$$

rolling the model forward: $\underbrace{s_{t+k+1} = f_{\phi}(s_{t+k},\, a_{t+k})}_{\text{each next state comes from the model}}$

- $a^{\star}_{t:t+H}$ is the *best* sequence; we assume the reward $r(s, a)$ is known (or use the learned $\mathcal{R}_{\phi}$)
- The hard part is the arg max: searching over many sequences. We will see practical search methods shortly

Planning: Putting It Together

- That is how a policy "plans through the model": imagine action sequences, roll them forward with f_ϕ, and pick the best-scoring one
- This is exactly what step 3 does, so we now have a complete algorithm
- But will planning with a *learned* model actually work? Let us see how it can break...

How can this approach fail?

How can this approach fail?

- In the first step, when we sample using random policy we may get get trajectories like the red line.

How can this approach fail?

- In the first step, when we sample using random policy we may get get trajectories like the red line.
- The agent may learn that going right means that we can go higher!

How can this approach fail?

- In the first step, when we sample using random policy we may get get trajectories like the red line.
- The agent may learn that going right means that we can go higher!
- Thus, it will learn an incomplete policy and fall at the top.

How can this approach fail?

- In the first step, when we sample using random policy we may get get trajectories like the red line.
- The agent may learn that going right means that we can go higher!
- Thus, it will learn an incomplete policy and fall at the top.
- The model was only trained where the *random* policy went, but the planner now wants to go somewhere the model has never seen

Why? A Distribution Mismatch

- Write $p_\pi(s)$ for **how often the agent visits state s when it follows policy π** (its distribution over states)

Why? A Distribution Mismatch

- Write $p_\pi(s)$ for **how often the agent visits state** s **when it follows policy** π (its distribution over states)
- π_0 (the policy that **collected the data**, e.g. the random policy) spends its time near the *bottom* of the cliff
- π_f (the **final** policy after planning) wants to climb to the *top*

Why? A Distribution Mismatch

- Write $p_\pi(s)$ for **how often the agent visits state s when it follows policy π** (its distribution over states)
- π_0 (the policy that **collected the data**, e.g. the random policy) spends its time near the *bottom* of the cliff
- π_f (the **final** policy after planning) wants to climb to the *top*
- So the states shift from the *data* distribution to the *learned-policy* distribution:

$$\underbrace{p_{\pi_0}(s)}_{\text{where we trained}} \neq \underbrace{p_{\pi_f}(s)}_{\text{where we now act}}$$

- The model is accurate on p_{π_0} but gets asked to predict on p_{π_f}, exactly where it never saw data
- Remember this picture; we return to it for *offline RL*

- How might we alleviate this issue?

- How might we alleviate this issue?
- Let's modify the algorithm a bit
  1. Run some policy (e.g. random policy) to collect data $D = \{(s, a, r, s')_i\}$
  2. Learn the model $f_\phi(s, a)$ (weights ϕ) to minimize $\sum_i \| f_\phi(s_i, a_i) - s'_i \|$
  3. Iteratively sample action sequences, run through model $f_\phi(s, a)$ to choose actions

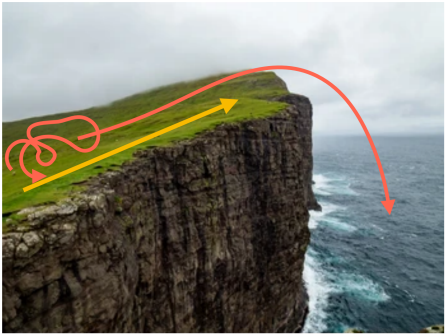
Model-Based RL

- How might we alleviate this issue?
- Let's modify the algorithm a bit
  1. Run some policy (e.g. random policy) to collect data $D = \{(s, a, r, s')_i\}$
  2. Learn the model $f_\phi(s, a)$ (weights ϕ) to minimize $\sum_i \| f_\phi(s_i, a_i) - s'_i \|$
  3. Iteratively sample action sequences, run through model $f_\phi(s, a)$ to choose actions
  4. Execute planned actions, appending visiting tuples (s, a, r, s') to D
  5. Go to step 2

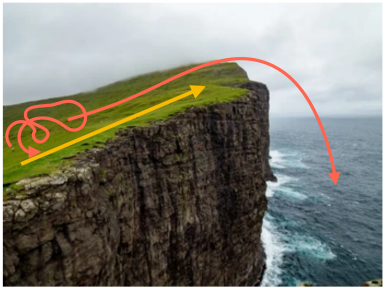
Model-Based RL

- How might we alleviate this issue?
- Let's modify the algorithm a bit
  1. Run some policy (e.g. random policy) to collect data $D = \{(s, a, r, s')_i\}$
  2. Learn the model $f_\phi(s, a)$ (weights ϕ) to minimize $\sum_i \| f_\phi(s_i, a_i) - s'_i \|$
  3. Iteratively sample action sequences, run through model $f_\phi(s, a)$ to choose actions
  4. Execute planned actions, appending visiting tuples (s, a, r, s') to D
  5. Go to step 2
- Now, we update the model using data collected with planning
- Then, replan periodically to help account for mistakes.

Revisiting the Cliff

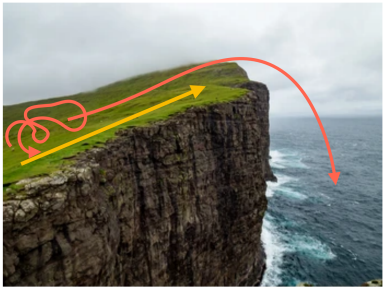


Revisiting the Cliff



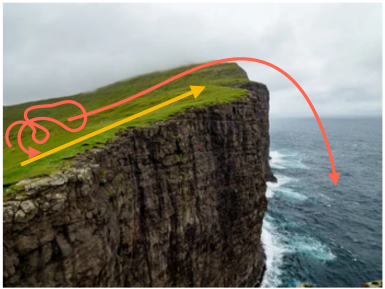
- Initially, its still the same. The agent learns that going right means that we can go higher!

Revisiting the Cliff



- Initially, its still the same. The agent learns that going right means that we can go higher!
- Then we re-train the model using the new observations

Revisiting the Cliff



- Initially, its still the same. The agent learns that going right means that we can go higher!
- Then we re-train the model using the new observations
- The final policy is different now. It learns to go to the top and stop.

Back to Planning: How to Search?

- Recall: planning = pick the action sequence whose model roll-out scores highest. The hard part was the arg max over sequences
- The fix we just used (replan + collect data) also relied on this search
- Two separate questions: first, how to *search* for a plan (random shooting, CEM); then, how often to *replan* (MPC)

How to Search? Random Shooting

- Simplest idea (no gradients, just guess):
  1. Sample N action sequences $a_{t:t+H}^{(1)}, \ldots, a_{t:t+H}^{(N)}$ from some distribution
  2. Roll each one through the model f_ϕ, sum the predicted rewards
  3. Keep the sequence with the highest predicted return

How to Search? Random Shooting

- Simplest idea (no gradients, just guess):
  1. Sample N action sequences $a_{t:t+H}^{(1)}, \ldots, a_{t:t+H}^{(N)}$ from some distribution
  2. Roll each one through the model f_ϕ, sum the predicted rewards
  3. Keep the sequence with the highest predicted return
- Derivative-free and trivially parallel
- This is the planner in **Nagabandi et al.**: "random shooting", no ensembles
- Weakness: most samples land in low-reward regions, wasteful

Cross-Entropy Method (CEM)

- Fix random shooting's waste: *refit* the sampling distribution toward the good sequences

Cross-Entropy Method (CEM)

- Fix random shooting's waste: *refit* the sampling distribution toward the good sequences
  1. Sample N sequences from a Gaussian $\mathcal{N}(\mu, \Sigma)$
  2. Evaluate under the model; keep the top-k *elites*
  3. Refit μ, Σ to the elites
  4. Repeat for a few iterations
- The distribution concentrates on high-return plans
- This is the optimizer hiding inside step 3 of our algorithm

A Different Question: When to Replan?

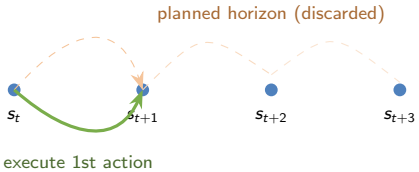
- Random shooting and CEM answered *how to find* a good plan
- MPC answers a *different* question: not how to *search*, but **how much of a plan to trust before re-deciding**
- It is a planning *technique*, not another search method

Model Predictive Control (MPC)

- A full open-loop plan is fragile: model errors compound over the horizon H

Model Predictive Control (MPC)

- A full open-loop plan is fragile: model errors compound over the horizon H
- **MPC**: plan a horizon, but *execute only the first action*, then re-plan from the state you actually land in



Model Predictive Control (MPC)

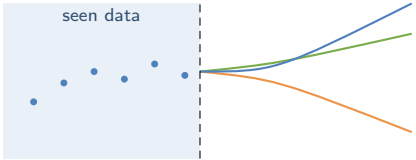
- A full open-loop plan is fragile: model errors compound over the horizon H
- **MPC**: plan a horizon, but *execute only the first action*, then re-plan from the state you actually land in
- This *is* the "execute, then re-plan" loop we used to fix the cliff, now it has a name
- Next: combine a CEM planner with an *ensemble* model $\Rightarrow$ **PETS**

Why Model Uncertainty Matters

- Recall the cliff: the model was *confidently wrong* off the data

Why Model Uncertainty Matters

- Recall the cliff: the model was *confidently wrong* off the data
- A single model gives one point prediction, no signal that it is guessing



off-data: the models *disagree*

Ensembles: Disagreement as Uncertainty

- Train an **ensemble** of N models (different initializations, bootstrapped data)

Ensembles: Disagreement as Uncertainty

- Train an **ensemble** of N models (different initializations, bootstrapped data)
- On data they *agree*; off data they *disagree*
- That disagreement measures **epistemic uncertainty** $u(s, a)$: a per state–action score for uncertainty due to *not having seen enough data*. It is high off-data, and it *shrinks* as you collect more (unlike the environment's inherent noise, which never does)
- Plan conservatively where $u(s, a)$ is high: don't trust a plan that exploits one model's fantasy

PETS: Planning with an Ensemble

- Put the two pieces together:
  - **Planner**: CEM (from the previous section)
  - **Model**: an ensemble of probabilistic dynamics models

PETS: Planning with an Ensemble

- Put the two pieces together:
  - **Planner**: CEM (from the previous section)
  - **Model**: an ensemble of probabilistic dynamics models
- Evaluate each candidate plan by propagating it through *sampled* models and averaging the return
- **PETS** (Chua et al., 2018): this is the "CEM-based planner" in the case study; Nagabandi's method is the same idea *without* the ensemble

Model-Based Policy Optimization

- We can also use model-based RL for policy optimization

Model-Based Policy Optimization

- We can also use model-based RL for policy optimization
- Key Idea:
  - Augment data with model-simulated roll-outs
  - This family (real data + model roll-outs) is **Dyna-style** (Sutton, 1991)

Model-Based Policy Optimization

- We can also use model-based RL for policy optimization
- Key Idea:
  - Augment data with model-simulated roll-outs
- Algorithm (**MBPO**; Janner et al., 2019):
  1. Collect data using current policy π_θ, add to D_{env}
  2. Update model $p_\phi(s' \mid s, a)$ using D_{env}
  3. Collect synthetic roll-outs using π_θ in model p_ϕ from states in D_{env}; add to D_{model}
  4. Update policy π_θ (and critic Q) using D_{model}
  5. Go to step 1

Model-Based Policy Optimization

- We can also use model-based RL for policy optimization
- Key Idea:
  - Augment data with model-simulated roll-outs
- Algorithm (**MBPO**; Janner et al., 2019):
  1. Collect data using current policy π_θ, add to D_{env}
  2. Update model $p_\phi(s' \mid s, a)$ using D_{env}
  3. Collect synthetic roll-outs using π_θ in model p_ϕ from states in D_{env}; add to D_{model}
  4. Update policy π_θ (and critic Q) using D_{model}
  5. Go to step 1
- Compatible with variety of model-free RL methods
- MBPO's design: *short, branched* roll-outs from *real* states keep model error from compounding

But We Go Back to the Env Anyway — Why Bother?

- Fair question: step 1 still collects real data. So why not skip the model and just take *more gradient steps* on the real replay buffer? Those are free too

But We Go Back to the Env Anyway — Why Bother?

- Fair question: step 1 still collects real data. So why not skip the model and just take *more gradient steps* on the real replay buffer? Those are free too
- Because the buffer only holds actions the *old* policy took
- A changing π_θ needs $Q(s, a)$ at the *new* actions $a \sim \pi_\theta$ — and there is no data there. The critic must extrapolate (we will name this failure in Part 2)

But We Go Back to the Env Anyway — Why Bother?

- Fair question: step 1 still collects real data. So why not skip the model and just take *more gradient steps* on the real replay buffer? Those are free too
- Because the buffer only holds actions the *old* policy took
- A changing π_θ needs $Q(s, a)$ at the *new* actions $a \sim \pi_\theta$ — and there is no data there. The critic must extrapolate (we will name this failure in Part 2)
- A buffer can only be **replayed**. A model can be **queried**: hand it (s, a_{new}), it hands back (r, s')

The model answers counterfactuals — "what if I had done this instead?" — which logged data cannot.

- The env visit is only to keep the model honest *where the policy now goes*; the model does the rest

Model-Based Policy Optimization

How to augment?

- Generate full trajectories from initial states?

Model-Based Policy Optimization

How to augment?

- Generate full trajectories from initial states?
  - Model may not be accurate for long horizons

Model-Based Policy Optimization

How to augment?

- Generate full trajectories from initial states?
  - Model may not be accurate for long horizons
- Generate partial trajectories from initial states?

Model-Based Policy Optimization

How to augment?

- Generate full trajectories from initial states?
  - Model may not be accurate for long horizons
- Generate partial trajectories from initial states?
  - May not get good coverage of later states

Model-Based Policy Optimization

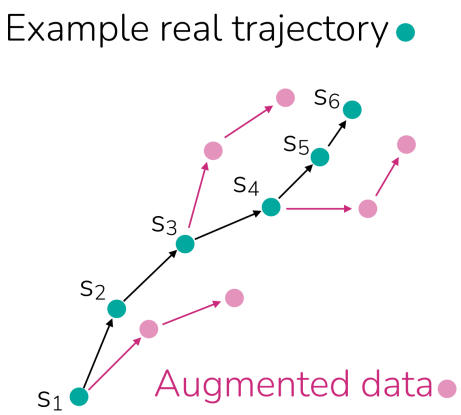
How to augment?

- Generate full trajectories from initial states?
  - Model may not be accurate for long horizons
- Generate partial trajectories from initial states?
  - May not get good coverage of later states
- Generate partial trajectories from all states in the data

Model-Based Policy Optimization

How to augment?

- Generate full trajectories from initial states?
  - Model may not be accurate for long horizons
- Generate partial trajectories from initial states?
  - May not get good coverage of later states
- Generate partial trajectories from all states in the data

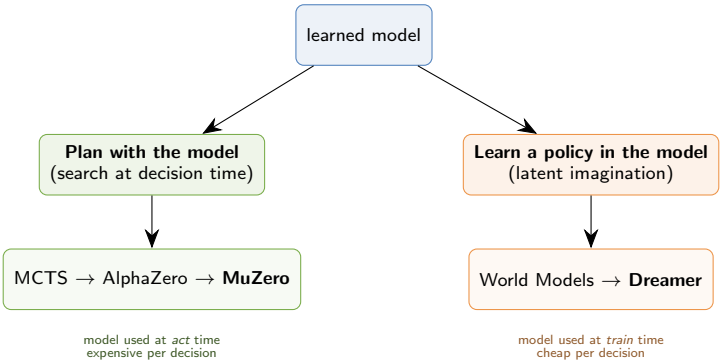


Modern Model-Based RL: Two Families

- Everything so far used the model to *plan*. Modern methods split along one axis:

Modern Model-Based RL: Two Families

- Everything so far used the model to *plan*. Modern methods split along one axis:



Planning with a Learned Model: AlphaZero & MuZero

- Both *plan* by **tree search** (MCTS), the discrete-move cousin of our CEM planner: grow a tree of moves, search the promising branches
- **AlphaZero**: MCTS + a *known* model, the rules of Go / chess handed to it for free
- **MuZero**: *learns* the model instead, and the clever part is *what* it learns: a model that predicts only **reward, value, and the best move**, never the raw pixels (a model of *what matters*, not what the world looks like)
- So the same search works *without being told the rules*: one architecture mastered Go, chess, shogi, *and* Atari (Schrittwieser et al., 2020)

Learning a Policy in the Model: Dreamer

- The other family learns a policy entirely *inside* a learned world model: "imagined" roll-outs, no real interaction per update
- Its model is trained to **reconstruct what the world looks like** (predict the next observation), the opposite choice from MuZero's "predict only what matters"
- **Dreamer / DreamerV3**: one fixed configuration solves 150+ tasks across Atari, continuous control, and Minecraft, with no per-task tuning

Two opposite bets on what a model is for: MuZero — don't waste capacity on pixels. Dreamer — pixels are a free, task-agnostic training signal. Both won.

When to use model-based RL?

- Advantages:
  - Models are immensely useful if easy to learn
  - Model can be trained without reward labels (self-supervised)
  - Model is somewhat task-agnostic (can sometimes be transferred across rewards)

When to use model-based RL?

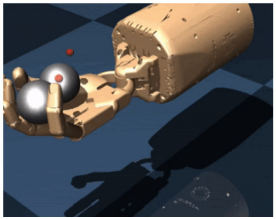
- Advantages:
  - Models are immensely useful if easy to learn
  - Model can be trained without reward labels (self-supervised)
  - Model is somewhat task-agnostic (can sometimes be transferred across rewards)
- Disadvantages:
  - Models don't optimize for task performance
  - Sometimes harder to learn than a policy

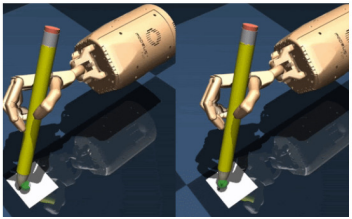
When to use model-based RL?

- Advantages:
  - Models are immensely useful if easy to learn
  - Model can be trained without reward labels (self-supervised)
  - Model is somewhat task-agnostic (can sometimes be transferred across rewards)
- Disadvantages:
  - Models don't optimize for task performance
  - Sometimes harder to learn than a policy

Whether to use a model depends on how hard it is to learn!

Case study: Model-Based RL for dexterous manipulation





Tasks: Baoding balls (left) and handwriting (right) on a simulated dexterous hand

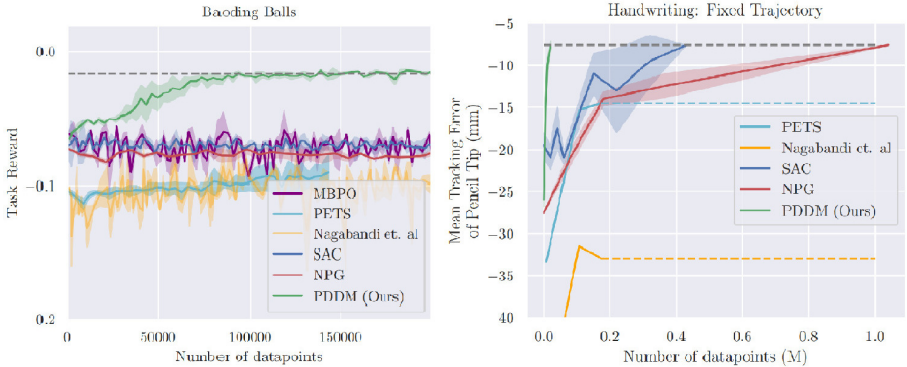
Model-free methods:

- **SAC**: actor–critic method
- **NPG**: policy gradient method

Model-based methods:

- **PDDM**: proposed method
- **MBPO**: RL with model-generated data
- **PETS**: CEM-based planner
- **Nagabandi et al.**: random shooting, no ensembles

Case study: Results



- Model-based **PDDM** (green) reaches higher reward and lower tracking error with *far less data* than the model-free baselines (SAC, NPG)

Nagabandi et al., *Deep Dynamics Models for Learning Dexterous Manipulation*

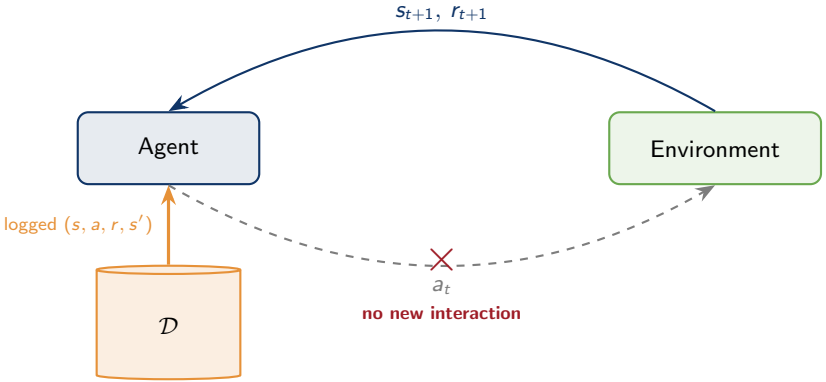
A Hidden Assumption

- Every algorithm in this course has shared one quiet privilege: **if the agent acts badly, it can simply try again**

A Hidden Assumption

- Every algorithm in this course has shared one quiet privilege: **if the agent acts badly, it can simply try again**
- DQN, PPO, DDPG, SAC, MBPO: all interleave *learning* with *acting*
- Even model-based RL fixed its early mistakes by going back out and collecting more data
- What if there is no "going back out"?

Offline RL: The Setting



- Given: a **fixed** dataset $\mathcal{D} = \{(s_i, a_i, r_i, s_i')\}$, collected by an unknown *behavior policy* π_β
- Goal: the best possible policy, **without a single new interaction**

Why Would We Tie Our Own Hands?

- Healthcare: we cannot experiment on patients
- Autonomous driving: we cannot crash in order to learn
- Recommenders, industrial control: years of logs, no appetite for live exploration
- The situation of modern machine learning: **mountains of logged data, zero permission to explore**

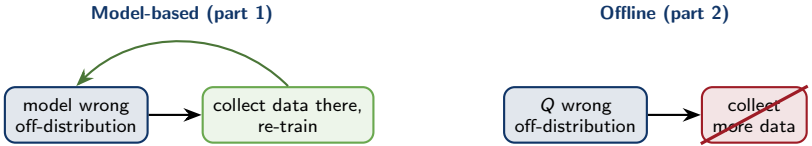
The Cliff, One Last Time

- Part 1's fix for distribution mismatch was to *execute*, *observe*, *re-train*: go collect data where the model was wrong

The Cliff, One Last Time

- Part 1's fix for distribution mismatch was to *execute*, *observe*, *re-train*: go collect data where the model was wrong
- Offline RL is **exactly this problem, with the fix forbidden**

Same Disease, Different Organ



- Careful, the failure *mechanisms* differ:
- The cliff was a **dynamics-model** error: f_ϕ predicted the wrong next state
- Offline RL's core failure is **value extrapolation**: Q invents values for actions the data never contains; it bites *even with perfect dynamics*
- Same disease (distribution shift), different organ (the model vs. the value function)

Why Not Just Copy the Data?

- Simplest possible idea: treat $\mathcal{D}$ as a supervised dataset and *imitate* it
- **Behavior cloning** (BC): train a policy π_θ to reproduce the data's actions:

$$\min_\theta \; -\mathbb{E}_{(s,a)\sim\mathcal{D}}\big[\log \pi_\theta(a \mid s)\big]$$

- In words: at each state in $\mathcal{D}$, push π_θ to put *high probability on the action the data actually took*, i.e. $\max_\theta \mathbb{E}[\log \pi_\theta]$. Papers write this as "min of $-\log$" (the standard *negative log-likelihood loss*), since optimizers minimize losses

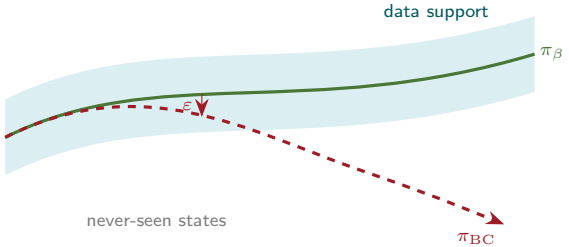
Why Not Just Copy the Data?

- Simplest possible idea: treat $\mathcal{D}$ as a supervised dataset and *imitate* it
- **Behavior cloning** (BC): train a policy π_θ to reproduce the data's actions:

$$\min_\theta \; -\mathbb{E}_{(s,a)\sim\mathcal{D}}\big[\log \pi_\theta(a \mid s)\big]$$

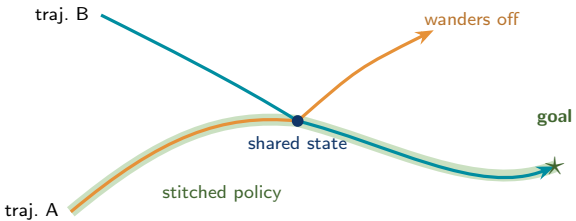
- In words: at each state in $\mathcal{D}$, push π_θ to put *high probability on the action the data actually took*, i.e. $\max_\theta \mathbb{E}[\log \pi_\theta]$. Papers write this as "min of $-\log$" (the standard *negative log-likelihood loss*), since optimizers minimize losses
- Plain supervised learning (like model learning, but for the policy): input s, target a, with no rewards, no Bellman, no bootstrapping. What could go wrong?

Behavior Cloning: The Ceiling and the Drift



- π_β = the (unknown) policy that *collected* the data (β for "behavior"); π_{BC} = the clone we train
- Ceiling: at best π_{BC} *matches* π_β, copying its mistakes too
- Drift: each step the clone has a small chance ε of a wrong action $\rightarrow$ a slightly unfamiliar state $\rightarrow$ a bigger error $\rightarrow$ off the data entirely
- These errors *compound*: total error grows like $O(\varepsilon T^2)$ over a horizon of T steps (Ross & Bagnell, 2010)

The Promise: Stitching



- $\mathcal{D}$ may contain **no good trajectory**, yet contain all the *pieces* of one
- Trajectories cross at a shared state: dynamic programming can **stitch** the good halves together
- The stitched policy is *better than anything in the data*; BC can never do this
- And we already own an off-policy algorithm: Q-learning learns from *any* data… right?

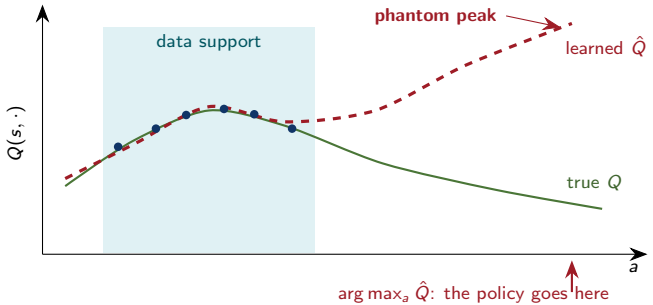
The Core Failure: Extrapolation Error

- Stitching needs **Q-learning**, so just run it on the fixed dataset. Look at its update target, which bootstraps from a max over actions:

$$y = r + \gamma \max_{a'} Q_\theta(s', a')$$

- That max ranges over **all** actions, including ones the dataset *never tried* (*out-of-distribution*, or **OOD**). Why is that fatal?

The Core Failure: Extrapolation Error



- Q_θ is a neural net: feed it *any* action and it returns a number, even where there is **no data**; there the value is not learned, only *extrapolated* ("invented")
- The max seeks the *largest* invented value $\Rightarrow$ **overestimation**; the policy $\arg\max_a \hat{Q}$ chases that *phantom*, and bootstrapping spreads the inflated value onward

Why Online RL Never Noticed

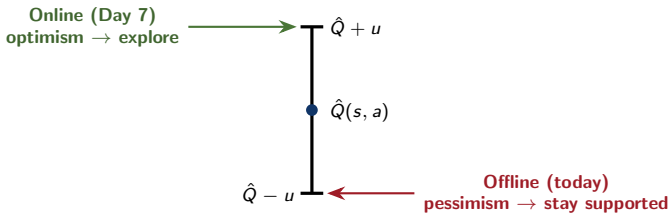
- Online, an overestimated action gets *tried*, disappoints, and is corrected; overestimation is self-erasing

Why Online RL Never Noticed

- Online, an overestimated action gets *tried*, disappoints, and is corrected; overestimation is self-erasing
- Offline, the policy can chase the phantom forever: no data will ever contradict it
- More gradient steps make it **worse**, not better: we optimize against the fantasy (Fujimoto et al., 2019: *extrapolation error*)

Online, hype meets reality. Offline, it never does.

The Fix: Optimism's Mirror Twin



- Day 7's rule: *in the face of uncertainty, be optimistic*: UCB acts on $\hat{Q} + u$, because uncertainty was an **opportunity**
- Offline, uncertainty can never be resolved: it is a **hazard**
- Same machinery, opposite sign: act on $\hat{Q} - u$, *pessimism* in the face of uncertainty
- Two places to install it: in the **values** (CQL) or in the **policy** (BCQ, IQL)

Conservative Q-Learning (CQL)

- Train Q with an extra penalty that makes unfamiliar (OOD) actions *look bad on purpose*

$$\min_Q \quad \alpha\, \mathbb{E}_{s\sim\mathcal{D}}\Big[\underbrace{\mathbb{E}_{a\sim\pi(\cdot|s)}[Q(s,a)]}_{\substack{\text{what } Q \textit{ claims } \pi \\ \text{can get (often OOD)}}} - \underbrace{\mathbb{E}_{a\sim\pi_\beta(\cdot|s)}[Q(s,a)]}_{\substack{\text{what } Q \text{ says about} \\ \text{actions we really saw}}}\Big] + \mathcal{L}_{\text{Bellman}}$$

- $\min_Q$ = train Q (tune its weights) to make this whole expression small. Both expectations are at the *same* s; π_β is the behavior policy from before — the actions the dataset actually logged at s

Conservative Q-Learning (CQL)

- Train Q with an extra penalty that makes unfamiliar (OOD) actions *look bad on purpose*

$$\min_Q \ \alpha\, \mathbb{E}_{s\sim\mathcal{D}}\Big[\underbrace{\mathbb{E}_{a\sim\pi(\cdot|s)}[Q(s,a)]}_{\substack{\text{what } Q \textit{ claims } \pi \\ \text{can get (often OOD)}}} - \underbrace{\mathbb{E}_{a\sim\pi_\beta(\cdot|s)}[Q(s,a)]}_{\substack{\text{what } Q \text{ says about} \\ \text{actions we really saw}}}\Big] + \mathcal{L}_{\text{Bellman}}$$

- Reads as: *"how much better does Q claim I can do by leaving the data?"* — a per-state measure of the phantom
- α = how hard to push (α=0 is plain Q-learning); $\mathcal{L}_{\text{Bellman}}$ = the normal TD loss, fitting Q to $r + \gamma \max_{a'} Q(s', a')$

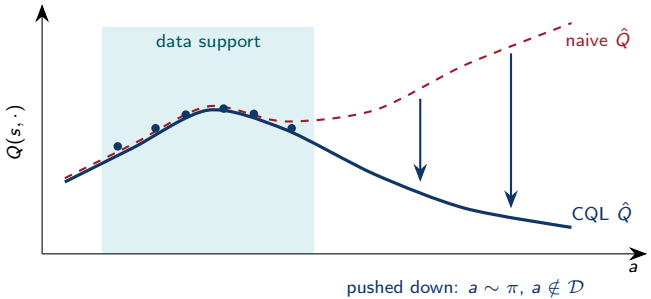
Conservative Q-Learning (CQL)

- Train Q with an extra penalty that makes unfamiliar (OOD) actions *look bad on purpose*

$$\min_Q \ \alpha \mathbb{E}_{s\sim\mathcal{D}}\Big[\underbrace{\mathbb{E}_{a\sim\pi(\cdot|s)}[Q(s,a)]}_{\substack{\text{what } Q \textit{ claims } \pi \\ \text{can get (often OOD)}}} - \underbrace{\mathbb{E}_{a\sim\pi_\beta(\cdot|s)}[Q(s,a)]}_{\substack{\text{what } Q \text{ says about} \\ \text{actions we really saw}}}\Big] + \mathcal{L}_{\text{Bellman}}$$

- Reads as: *"how much better does Q claim I can do by leaving the data?"* — a per-state measure of the phantom
- **Why the subtraction?** Lowering Q *everywhere* changes no arg max, so it changes no policy. The gap is shift-invariant: it can only shrink by *tilting* Q down on OOD actions *relative to* in-data ones
- If $\pi = \pi_\beta$ the penalty is 0: no punishment for staying home

What CQL Does to the Phantom

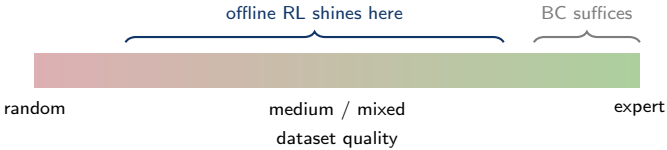


- The phantom peak from before, after CQL training
- OOD actions now look *worse* than in-data actions: the greedy policy stays home

A Second Family: Constrain the Policy

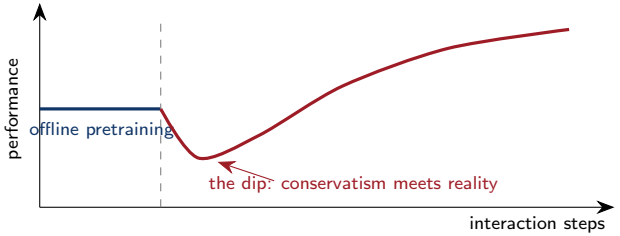
- CQL enforced pessimism in the **values**. A second family enforces it in the **policy** instead:
- **Keep the policy close to the data**: let it pick only actions the behavior policy plausibly could have taken, so it never reaches an OOD action in the first place
- **BCQ** and **IQL** are the well-known instances; they differ in *how* they enforce this, and those details are genuinely involved (beyond our scope)
- Same principle as CQL, applied in a different place: *stay where the data is*

How the Field Keeps Score



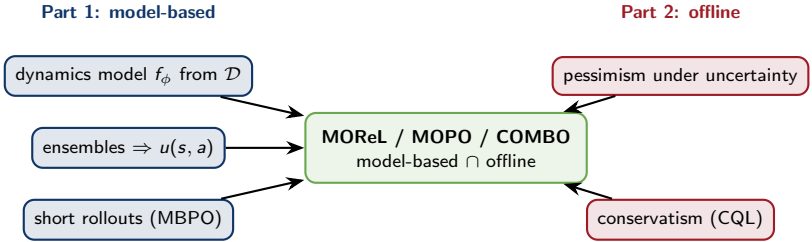
- Offline RL is measured on shared **benchmark suites** of fixed datasets: the standard one is **D4RL** ("Datasets for Deep Data-Driven RL")
- Its tasks stress different skills: e.g. a maze-navigation set built to *require stitching*, or a dexterous-hand set of *human demonstrations*
- Each task comes at several *data qualities* (random / medium / expert mixtures), and method rankings flip between them
- Scores are normalized (0 = random, 100 = expert); on expert data plain BC is hard to beat, so **always run it as the baseline**

Offline → Online: The Deployment Recipe



- In practice: **pretrain offline** (data is free) → **fine-tune online** (interaction is scarce and expensive)
- Naive fine-tuning dips first: the pessimistic values get violently re-calibrated before they improve
- Fixes keep the offline ballast while admitting new data: AWAC, Cal-QL, RLPD
- You already know this recipe's most famous instance: *pretrain on logged human data, then fine-tune with RL*, the LLM pipeline (Day 9!)

Closing the Loop: Model-Based Offline RL



- Learn a model from $\mathcal{D} \rightarrow$ it is wrong off-support (the cliff!) $\rightarrow$ measure that with ensemble disagreement $\rightarrow$ build a *pessimistic MDP* $\rightarrow$ run MBPO inside it
- **MOReL**, **MOPO**, **COMBO** are all instances of *one* idea: learn a model from the batch, then *subtract a penalty wherever the model is uncertain* (e.g. reward $\hat{r} - \lambda\, u(s,a)$). This is pessimism, now about the *model*

One Story, Two Signs

- Exploration (Day 7) and offline RL (today) are the **same OOD problem, in reverse**
- Online: uncertainty is an opportunity, so add the bonus and go look
- Offline: uncertainty is a hazard, so subtract the bonus and stay home

Optimism for exploration, pessimism for offline data: the sign of the uncertainty bonus is the whole story.

Summary

- **Model-based RL**: learn the dynamics (and/or reward) by supervised learning, then *plan* (random shooting, CEM, MPC) or *optimize a policy* (Dyna-style, MBPO) with the model
- Models are wrong off-distribution: **ensembles** measure that, and re-collecting data (MPC) fixes it
- Modern lines: planning with a learned model (MuZero) and latent imagination (Dreamer)
- **Offline RL**: learn from a fixed batch, no interaction. The core failure is *value extrapolation* on OOD actions; the fix is **pessimism** (CQL, BCQ, IQL)
- The two halves meet in model-based offline RL (MOReL / MOPO / COMBO): a model plus a penalty for where it is uncertain
- One spine: *optimism* explores, *pessimism* stays safe; the sign of the uncertainty bonus is the whole story

Day 9: RL in the Real World (RLHF).

- RLHF's *reward model* is the learned reward model $\mathcal{R}_\phi$ from the start of today
- Learning a policy from fixed human demonstration / preference data is an **offline** setting
- "Pretrain on logged data, then fine-tune with RL" is exactly the offline-to-online recipe, now applied to LLMs

References

These slides have been adapted from

- Chelsea Finn & Karol Hausman, Stanford CS224R: Deep Reinforcement Learning
- Sergey Levine, Berkeley CS285 Deep Reinforcement Learning
- David Silver, Deepmind: Introduction to Reinforcement Learning

References: Key Papers (Model-Based)

- Sutton (1991). *Dyna, an integrated architecture for learning, planning and reacting.*
- Chua et al. (2018). *Deep RL in a handful of trials* (PETS). arXiv:1805.12114
- Nagabandi et al. (2019). *Deep dynamics models for learning dexterous manipulation* (PDDM). arXiv:1909.11652
- Janner et al. (2019). *When to trust your model* (MBPO). arXiv:1906.08253
- Silver et al. (2018). *A general RL algorithm that masters chess, shogi and Go through self-play* (AlphaZero). *Science* 362(6419).
- Schrittwieser et al. (2020). *Mastering Atari, Go, chess and shogi by planning with a learned model* (MuZero). arXiv:1911.08265
- Hafner et al. (2023). *Mastering diverse domains through world models* (DreamerV3). arXiv:2301.04104

References: Key Papers (Offline RL)

- Ross & Bagnell (2010). *Efficient reductions for imitation learning* (compounding error).
- Fujimoto et al. (2019). *Off-policy deep RL without exploration* (BCQ; extrapolation error). arXiv:1812.02900
- Kumar et al. (2020). *Conservative Q-learning for offline RL* (CQL). arXiv:2006.04779
- Kidambi et al. (2020) *MOReL*; Yu et al. (2020) *MOPO*; Yu et al. (2021) *COMBO*.
- Kostrikov et al. (2021). *Offline RL with implicit Q-learning* (IQL). arXiv:2110.06169
- Fu et al. (2020). *D4RL: datasets for deep data-driven RL*. arXiv:2004.07219
- Offline→online fine-tuning: Nair et al. (2020) *AWAC*; Nakamoto et al. (2023) *Cal-QL*; Ball et al. (2023) *RLPD*.